



CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

Fundamentos de Programação

Lógica · Algoritmos · Variáveis · Estruturas de Controle · Vetores e Matrizes · Funções
Exemplos práticos em Python



Sumário

Capítulo 1 — Lógica, Algoritmos e Linguagens de Programação

Capítulo 2 — Variáveis, Tipos de Dados e Entrada/Saída

Capítulo 3 — Estrutura Condicional

Capítulo 4 — Estrutura de Repetição

Capítulo 5 — Vetores e Matrizes

Capítulo 6 — Modularização e Funções



Capítulo 1 — Lógica, Algoritmos e Linguagens de Programação

Antes de escrever qualquer código, todo programador precisa aprender a pensar de forma lógica e organizada. Este capítulo apresenta os conceitos de lógica de programação e algoritmo, mostra diferentes formas de representar um algoritmo e introduz o papel das linguagens de programação — inclusive comparando um mesmo problema resolvido em quatro linguagens diferentes.

1.1 Lógica de programação

Lógica de programação é o conjunto de técnicas usadas para organizar o raciocínio e criar sequências lógicas de instruções capazes de resolver um problema. Ela independe da linguagem que será usada depois — é a base para qualquer pessoa que deseje programar, em qualquer tecnologia.

Pensar a lógica de um problema envolve três etapas principais:

- Entender claramente o problema a ser resolvido;
- Planejar uma solução, dividindo o problema em passos menores;
- Organizar esses passos em uma ordem lógica e executável.

1.2 Algoritmo

Um algoritmo é uma sequência finita e ordenada de passos que resolve um problema ou realiza uma tarefa. Esse conceito é anterior à computação: uma receita de bolo, por exemplo, também é um algoritmo — os ingredientes são os dados de entrada, o modo de preparo são as instruções, e o bolo pronto é a saída.

Características de um bom algoritmo

- **Finitude** — precisa ter início, meio e fim, com número finito de passos;
- **Definição (clareza)** — cada passo deve ser claro, sem ambiguidade;
- **Entrada** — pode receber zero ou mais dados antes de começar;
- **Saída** — deve produzir ao menos um resultado;
- **Efetividade** — cada passo deve ser simples e realmente executável.

Formas de representar um algoritmo

- **Descrição narrativa** — escrita em linguagem natural, passo a passo;
- **Fluxograma** — representação gráfica com símbolos padronizados;
- **Pseudocódigo** — escrito em português estruturado, próximo do código real;
- **Código-fonte** — o algoritmo já traduzido para uma linguagem de programação.

1.3 Linguagens de programação

Uma linguagem de programação é um conjunto de regras (sintaxe) que permite escrever instruções que o computador consegue executar. Existem centenas delas — Python, C, Java, JavaScript, C++, Go, Ruby, entre outras — cada uma com vantagens diferentes conforme o tipo de aplicação.



Baixo nível x alto nível

Linguagens de baixo nível estão mais próximas da linguagem de máquina (como o Assembly): são difíceis de ler, mas oferecem alto desempenho e controle direto do hardware. Linguagens de alto nível, como Python e Java, estão mais próximas da linguagem humana: são mais fáceis de escrever e entender. Quanto maior o nível de abstração, mais simples costuma ser programar.

Compiladores x interpretadores

- **Compilador** — traduz todo o código de uma vez e gera um arquivo executável (ex.: C);
- **Interpretador** — lê e executa o código linha por linha, sem gerar executável (ex.: Python);
- **Linguagens híbridas** — Java, por exemplo, compila para bytecode, que depois é interpretado pela JVM.

1.4 Um mesmo problema, quatro linguagens

Para perceber como a lógica é a mesma independentemente da sintaxe, veja abaixo um programa simples — que lê dois números e mostra a soma — escrito em quatro linguagens diferentes.

Python (interpretada)

```
a = float(input("1o número: "))
b = float(input("2o número: "))
soma = a + b
print(f"A soma é {soma}")
```

C (compilada)

```
#include <stdio.h>
int main() {
    float a, b, soma;
    scanf("%f %f", &a, &b);
    soma = a + b;
    printf("Soma: %.2f", soma);
    return 0;
}
```

Java (compilada para bytecode / JVM)

```
import java.util.Scanner;
public class Soma {
    public static void main(String[] x) {
        Scanner sc = new Scanner(System.in);
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        System.out.println("Soma: " + (a + b));
    }
}
```

JavaScript (interpretada pelo navegador / Node.js)

```
let a = parseFloat(prompt("1o número:"));
let b = parseFloat(prompt("2o número:"));
let soma = a + b;
console.log("Soma: " + soma);
```



1.5 Aplicações da programação

A lógica de programação é a base para praticamente qualquer área da tecnologia atual:

- Desenvolvimento web e aplicativos mobile;
- Jogos digitais e realidade virtual;
- Inteligência artificial e ciência de dados;
- Automação de processos (RPA) e robótica;
- Sistemas embarcados, IoT e sistemas operacionais.

Independentemente da área escolhida, tudo começa pela mesma base: uma lógica de programação bem estruturada.



Capítulo 2 — Variáveis, Tipos de Dados e Entrada/Saída

Depois de compreender a lógica por trás de um algoritmo, o próximo passo é aprender como um programa guarda e manipula informações. Este capítulo apresenta o conceito de variável, os principais tipos de dados em Python e como um programa troca informações com o usuário através de entrada (input) e saída (print) de dados.

2.1 Variáveis

Uma variável é um espaço reservado na memória do computador para armazenar um valor. Esse valor pode mudar durante a execução do programa — daí o nome “variável”. Toda variável tem um nome (identificador) e um valor associado; uma boa forma de visualizar isso é imaginar uma caixa com etiqueta: a etiqueta é o nome e o conteúdo é o valor guardado.

```
idade = 20      # a caixa "idade" guarda o valor 20
idade = 21      # o valor pode ser alterado depois
```

Regras para nomear variáveis em Python

- Deve começar com letra ou underscore (_) — nunca com número;
- Não pode conter espaços ou símbolos especiais (@, #, -, etc.);
- Python diferencia maiúsculas de minúsculas (idade é diferente de Idade);
- Evite usar palavras reservadas da linguagem (if, for, while...);
- Boa prática: utilize nomes descritivos, como idade ou nome_aluno.

2.2 Tipos de dados primitivos

Python define automaticamente o tipo de uma variável de acordo com o valor atribuído a ela. Os quatro tipos primitivos mais usados são:

Tipo	Descrição	Exemplo
int	número inteiro, positivo ou negativo	idade = 25
float	número com casas decimais	altura = 1.75
str	cadeia de caracteres (texto)	nome = "Ana"
bool	valor lógico: True ou False	ativo = True

A função `type()` mostra qual é o tipo de dado de um valor ou variável:

```
print(type(10))      # <class 'int'>
print(type(3.14))    # <class 'float'>
print(type("Python")) # <class 'str'>
print(type(True))    # <class 'bool'>
```

2.3 Conversão de tipos (casting)

As funções `int()`, `float()`, `str()` e `bool()` convertem um valor de um tipo para outro. Isso é essencial, por exemplo, quando um valor digitado pelo usuário precisa ser usado em um cálculo.



```
idade = "20"
idade = int(idade)          # texto -> inteiro
altura = float("1.70")     # texto -> float
nota = str(9.5)            # número -> texto
```

2.4 Entrada de dados: input()

`input()` pausa o programa e aguarda o usuário digitar algo, pressionando Enter em seguida. É importante lembrar que **input()** sempre retorna um valor do tipo texto (**str**) — por isso, muitas vezes é necessário convertê-lo antes de usá-lo em cálculos.

```
nome = input("Digite seu nome: ")
idade = int(input("Digite sua idade: "))
print("Olá,", nome)
```

2.5 Saída de dados: print()

`print()` exibe informações na tela. É possível exibir vários valores separados por vírgula, alterar o separador padrão com o parâmetro `sep`, ou formatar a saída usando **f-strings**, que permitem inserir variáveis diretamente dentro de um texto.

```
nome = "Ana"
idade = 22
print("Nome:", nome, "Idade:", idade, sep=" | ")
print(f"{nome} tem {idade} anos")
print(f"Nota final: {9.5678:.2f}") # Nota final: 9.57
```

Resumo do capítulo: uma variável guarda um dado; `type()` mostra o seu tipo; `input()` e `print()` são as portas de entrada e saída de um programa — e nunca se deve esquecer que tudo o que vem do `input()` chega como texto.



Capítulo 3 — Estrutura Condicional (if / else)

Um programa realmente útil precisa tomar decisões. Este capítulo apresenta a estrutura condicional — a forma como um programa escolhe qual caminho seguir de acordo com uma condição verdadeira ou falsa — usando os comandos if e else.

3.1 Operadores relacionais e lógicos

Antes de usar condicionais, é preciso saber comparar valores. Os operadores relacionais comparam dois valores e resultam sempre em True ou False:

Operador	Significado	Exemplo
==	igual a	5 == 5 → True
!=	diferente de	5 != 3 → True
>	maior que	8 > 3 → True
<	menor que	8 < 3 → False
>=	maior ou igual a	5 >= 5 → True
<=	menor ou igual a	4 <= 3 → False

Os operadores lógicos permitem combinar duas ou mais condições em uma só verificação:

Operador	Significado	Exemplo
and	verdadeiro se ambas as condições forem verdadeiras	(5>3) and (2>1) → True
or	verdadeiro se ao menos uma condição for verdadeira	(5>3) or (1>2) → True
not	inverte o valor lógico da condição	not (5>3) → False

3.2 Estrutura if

O bloco indentado abaixo do if só é executado quando a condição é verdadeira. A indentação (espaços no início da linha) é o que define, em Python, o que pertence ao bloco condicional.

```
idade = 20
if idade >= 18:
    print("Você é maior de idade")
```

3.3 Estrutura if / else

O bloco do else executa apenas quando a condição do if é falsa — ou seja, if e else são caminhos mutuamente exclusivos.



```
numero = 7
if numero % 2 == 0:
    print("Par")
else:
    print("Ímpar")
```

3.4 If aninhado (nested if)

Um if aninhado é um if dentro de outro if, usado para tratar várias condições em cascata. Cada nível de aninhamento exige uma indentação a mais.

```
a, b, c = 5, 8, 3
if a > b:
    if a > c:
        print("a é o maior")
    else:
        print("c é o maior")
else:
    if b > c:
        print("b é o maior")
    else:
        print("c é o maior")
```

3.5 Exemplos práticos

```
# classificação de nota
nota = float(input("Digite a nota: "))
if nota >= 6:
    print("Aprovado")
else:
    print("Reprovado")
```

```
# cálculo de desconto
valor = float(input("Valor da compra: "))
if valor > 100:
    desconto = valor * 0.10
else:
    desconto = 0
print(f"Desconto: R$ {desconto:.2f}")
```

3.6 Erros comuns

- Usar = (atribuição) em vez de == (comparação);
- Esquecer os dois-pontos (:) no final da linha do if ou else;
- Indentação incorreta, ou misturar espaços com tabulações;
- Esquecer o else quando o programa precisa tratar o caso contrário.



Capítulo 4 — Estrutura de Repetição (while e for)

Estruturas de repetição — também chamadas de laços ou loops — permitem executar um bloco de código várias vezes sem reescrevê-lo. Este capítulo apresenta os dois principais laços de Python: `while` e `for`, com destaque especial para o `for`, o mais utilizado no dia a dia da linguagem.

4.1 Quando usar um laço de repetição

- Para processar todos os itens de uma lista;
- Para repetir uma tarefa um número específico de vezes;
- Para validar uma entrada até que o usuário digite algo correto;
- Para contar ou somar valores automaticamente.

4.2 Estrutura while

O `while` repete um bloco de código enquanto uma condição permanecer verdadeira. É fundamental atualizar essa condição dentro do laço — caso contrário, o programa entra em um laço infinito.

```
contador = 1
while contador <= 5:
    print(contador)
    contador += 1
```

Cuidado: se o programador esquecer de atualizar a variável de controle, a condição nunca se torna falsa e o laço nunca termina.

4.3 Estrutura for (a mais usada em Python)

O `for` percorre, um a um, os elementos de uma sequência — uma lista, uma string ou um `range`. É o laço ideal quando já sabemos o que será percorrido ou quantas vezes o bloco deve repetir.

```
for variavel in sequencia:
    instrução
```

A função range()

`range(início, fim, passo)` gera uma sequência de números inteiros. O valor de `fim` nunca é incluído na sequência.

```
range(5)           # 0, 1, 2, 3, 4
range(1, 6)        # 1, 2, 3, 4, 5
range(0, 10, 2)     # 0, 2, 4, 6, 8
```



Exemplos com for

```
# soma de 1 até 10
soma = 0
for i in range(1, 11):
    soma += i
print(f"Soma total: {soma}")
```

```
# tabuada do 7
numero = 7
for i in range(1, 11):
    print(f"{numero} x {i} = {numero * i}")
```

```
frutas = ["maçã", "banana", "uva"]
for fruta in frutas:
    print(fruta)
```

4.4 break e continue

- **break** interrompe o laço imediatamente;
- **continue** pula apenas a iteração atual e segue para a próxima.

```
for i in range(1, 11):
    if i == 5:
        break          # interrompe o laço
    print(i)

for i in range(1, 6):
    if i == 3:
        continue       # pula esta iteração
    print(i)
```

4.5 for x while — quando usar cada um

- **for** → quando já sabemos quantas vezes repetir, ou vamos percorrer uma sequência;
- **while** → quando repetimos até que uma condição seja satisfeita, sem saber quantas vezes de antemão.



Capítulo 5 — Vetores e Matrizes

Programas frequentemente precisam armazenar diversos valores relacionados — uma lista de notas, os nomes de uma turma, os preços de um estoque. Para isso existem os vetores (em Python, listas) e, quando os dados possuem duas dimensões, as matrizes.

5.1 O que é um vetor?

Um vetor é uma estrutura que armazena vários valores em uma única variável. Os valores ficam organizados em sequência, e cada um possui uma posição chamada índice. Em Python, um vetor é representado por uma lista (list), escrita entre colchetes.

```
numeros = [10, 20, 30, 40]
nomes = ["Ana", "Bruno", "Carla"]
```

Os índices sempre começam em **0**, e é possível também usar índices negativos para contar a partir do final da lista:

```
numeros = [10, 20, 30, 40]
print(numeros[0])    # 10 (primeiro elemento)
print(numeros[-1])   # 40 (último elemento)
```

Para percorrer todos os elementos, o mais comum é usar um laço for:

```
for n in numeros:
    print(n)
```

5.2 Métodos mais usados em listas

Método	O que faz	Exemplo
append(x)	adiciona x ao final da lista	frutas.append("uva")
remove(x)	remove a primeira ocorrência do valor x	frutas.remove("banana")
pop(i)	remove e retorna o item do índice i	frutas.pop(0)
len(lista)	retorna a quantidade de itens	len(numeros)
sort()	ordena a lista em ordem crescente	numeros.sort()
max() / min()	maior / menor valor da lista	max(numeros)

5.3 Exemplos práticos

```
# soma e média
notas = [7.5, 8.0, 6.5, 9.0]
soma = 0
for n in notas:
    soma += n
print(f"Média: {soma / len(notas):.2f}")
```



```
# buscar um elemento
alunos = ["Ana", "Bruno", "Carla"]
busca = input("Nome a buscar: ")
if busca in alunos:
    print("Aluno encontrado!")
else:
    print("Aluno não encontrado.")
```

5.4 Matriz (breve introdução)

Uma matriz é um “vetor de vetores”: organiza os dados em linhas e colunas, como uma tabela. Em Python, é representada por uma lista de listas. O primeiro índice acessa a linha; o segundo índice acessa a coluna.

```
matriz = [
    [1, 2, 3],
    [4, 5, 6]
]
print(matriz[0][1])  # 2 (linha 0, coluna 1)
print(matriz[1][2])  # 6 (linha 1, coluna 2)
```

Resumo: o vetor organiza dados em uma dimensão; a matriz organiza dados em duas (ou mais) dimensões — mas ambos são sempre acessados por índice, começando em zero.



Capítulo 6 — Modularização e Funções

À medida que um programa cresce, torna-se importante dividi-lo em partes menores e reutilizáveis. Este capítulo apresenta o conceito de modularização e como criar funções em Python usando `def`, parâmetros e `return`.

6.1 Modularização

Modularizar é dividir um programa grande em partes menores — módulos ou funções — cada uma responsável por uma tarefa específica. Isso traz vantagens importantes:

- Organização e clareza — cada função tem uma responsabilidade;
- Reutilização — evita repetir o mesmo código várias vezes;
- Facilidade de manutenção e correção de erros;
- Facilita o trabalho em equipe, dividindo tarefas;
- Permite testar cada parte do programa isoladamente.

6.2 Funções em Python

Uma função é um bloco de código nomeado que executa uma tarefa específica e pode ser chamado (executado) quantas vezes forem necessárias. Em Python, uma função é criada com a palavra-chave `def`.

```
def nome_da_funcao():  
    instrução  
  
nome_da_funcao()    # chamando a função
```

Parâmetros

Parâmetros permitem que uma função receba valores diferentes a cada chamada:

```
def saudacao(nome):  
    print(f"Olá, {nome}!")  
  
saudacao("Ana")  
saudacao("Bruno")
```

Retorno com `return`

`return` devolve um valor para quem chamou a função — diferente de `print()`, que apenas exibe algo na tela.

```
def dobro(numero):  
    return numero * 2  
  
resultado = dobro(5)  
print(resultado)    # 10
```



6.3 Mais exemplos

```
def somar(a, b):  
    return a + b  
  
print(somar(3, 4))    # 7
```

```
def par_ou_impar(numero):  
    if numero % 2 == 0:  
        return "Par"  
    else:  
        return "Ímpar"  
  
print(par_ou_impar(7))    # Ímpar
```

Parâmetros também podem ter um **valor padrão**, usado quando o argumento não é informado na chamada:

```
def saudacao(nome, mensagem="Bem-vindo"):  
    print(f"{mensagem}, {nome}!")  
  
saudacao("Ana")                # Bem-vindo, Ana!  
saudacao("Bruno", "Boa tarde") # Boa tarde, Bruno!
```

6.4 Escopo de variáveis

Variáveis criadas dentro de uma função são locais: só existem enquanto a função está executando e não afetam variáveis externas de mesmo nome.

```
x = 10    # variável global  
  
def teste():  
    x = 5    # variável local  
    print(x)  
  
teste()    # 5  
print(x)   # 10
```

Resumo do capítulo e do curso: a modularização organiza o programa em partes menores e reutilizáveis; def, parâmetros e return são a base para criar suas próprias funções. Do algoritmo até as funções, cada capítulo desta apostila constrói, passo a passo, a fundação da lógica de programação.